DATA STRUCTURES · DIGITAL NOTES

UNIT II

Linked Lists

Singly · Doubly · Circular linked lists · Arrays vs linked lists · Applications

Roadmap of this unit

2.1	Introduction – why linked lists?
2.2	Singly linked list: representation & operations
2.3	Doubly linked list
2.4	Circular linked list
2.5	Comparing arrays and linked lists
2.6	Applications of linked lists

2.1 Introduction to Linked Lists

DEFINITION
Linked list = a linear data structure made of **nodes**. Each node stores **data** and a **link (pointer)** to the next node. Nodes can lie **anywhere in memory**; the links join them in order.

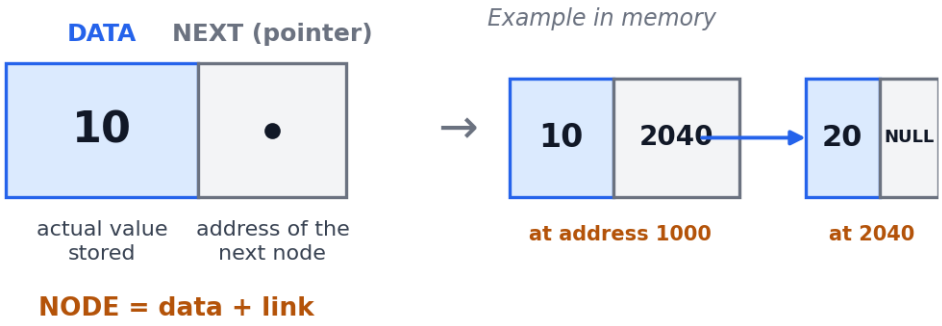


Fig: Structure of a node – data part and pointer part

Why do we need linked lists? (problems of arrays)

- Array size is **fixed** – wastes memory if too big, overflows if too small.
- Insert / delete in the middle needs **shifting** of many elements – O(n).
- Needs a **big continuous** block of free memory.
- Linked list: size **grows and shrinks at run time** (dynamic) and insertion needs only pointer changes.

Terminology

Term	Meaning
Node	One element = data + pointer(s)
head / start	Pointer to the **first** node – the only way to enter the list
next	Pointer to the following node
NULL	Special value meaning “no node” – marks the end
Traversal	Visiting each node by following next links

Types of linked lists

Type	Links per node	Direction	Last node points to
Singly	1 (next)	Forward only	NULL
Doubly	2 (prev, next)	Forward + backward	NULL
Circular	1 or 2	Forward (or both)	**First node**

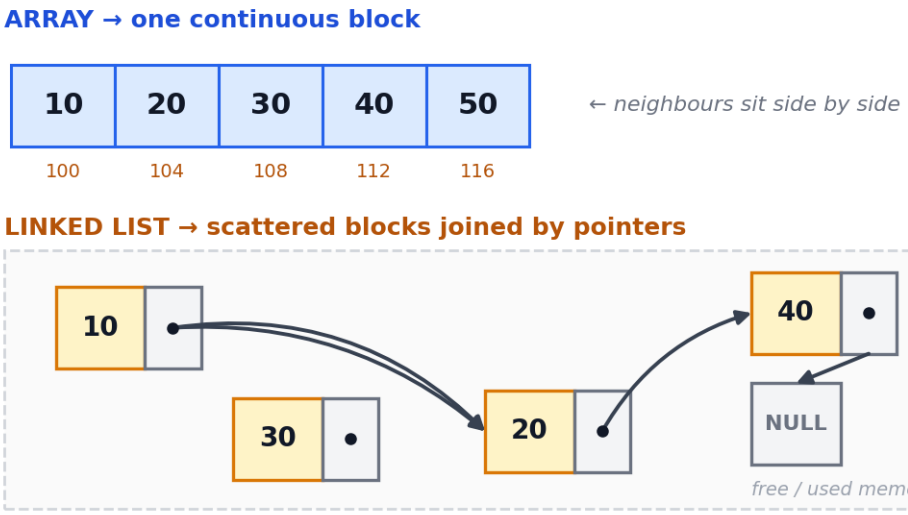


Fig: Array = continuous memory; linked list = scattered nodes connected by pointers

REMEMBER

- **Advantages:** dynamic size · easy insertion/deletion · no memory shifting · efficient memory use.
- **Disadvantages:** extra memory for pointers · **no direct access** (no a[i]) · sequential traversal only · poor cache use.

2.2 Singly Linked List (SLL)

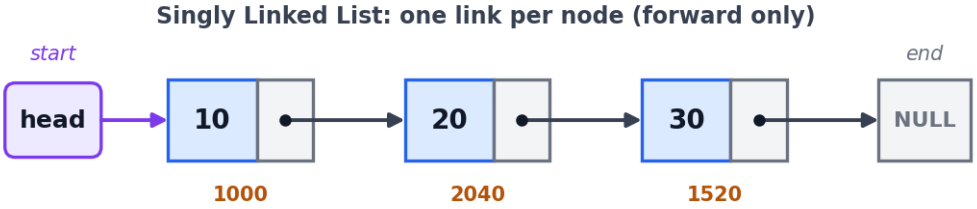


Fig: Singly linked list with addresses – head holds the address of the first node

Representation in C

```
struct Node {
    int data;              // value stored in the node
    struct Node *next;     // address of the next node (self-referential structure)
};
struct Node *head = NULL;  // empty list

struct Node* createNode(int x) {
    struct Node *n = (struct Node*) malloc(sizeof(struct Node));   // allocate
    n->data = x;  n->next = NULL;
    return n;
}
```

A node is a self-referential structure – it contains a pointer to its own type

Operation	Time	Remark
Insert at beginning	O(1)	No traversal needed
Insert at end	O(n)	O(1) if a tail pointer is kept
Insert after a given node	O(1)	Position already known
Delete first node	O(1)	Move head
Delete last node	O(n)	Need the second-last node
Search / access k-th	O(n)	Must walk from head

A. Traversal (display)

1. Set **temp = head**.
2. While **temp != NULL**: print temp→data, then **temp = temp→next**.
3. Stop when temp becomes NULL.

```
void display() {
    struct Node *temp = head;
    while (temp != NULL) { printf("%d -> ", temp->data); temp = temp->next; }
    printf("NULL\n");
}
```

B. Insertion

1) At the beginning

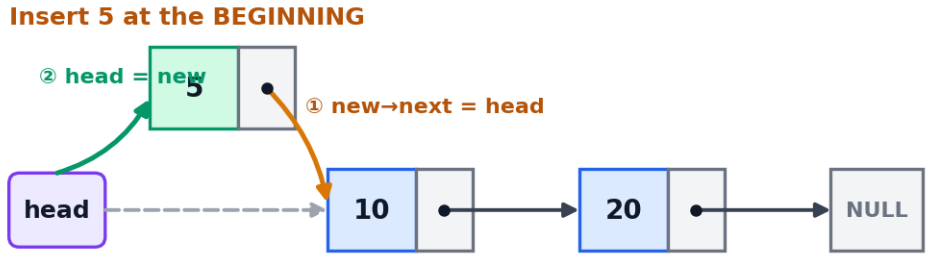


Fig: Insertion at the beginning – only two pointer changes

```
void insertBegin(int x) {
    struct Node *n = createNode(x);
    n->next = head;      // (1) new node points to old first node
    head = n;            // (2) head now points to new node
}
```

2) In the middle (after a given position)

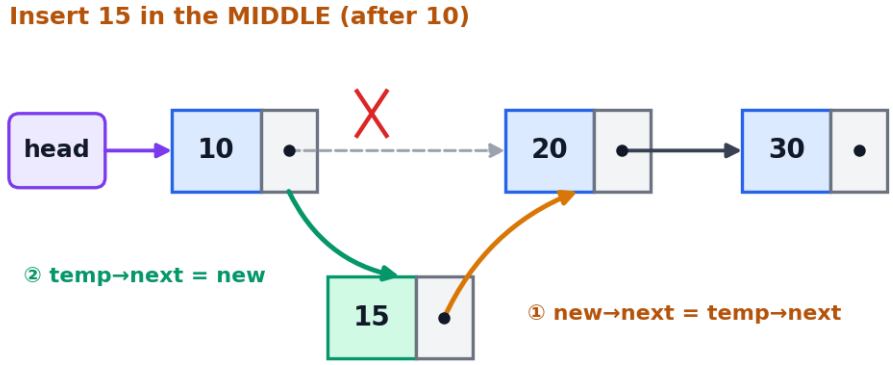


Fig: Insertion after node 10 – order of the two steps matters!

COMMON MISTAKE
Do **step ① first** (new→next = temp→next). If you do ② first, you lose the address of the rest of the list.

3) At the end

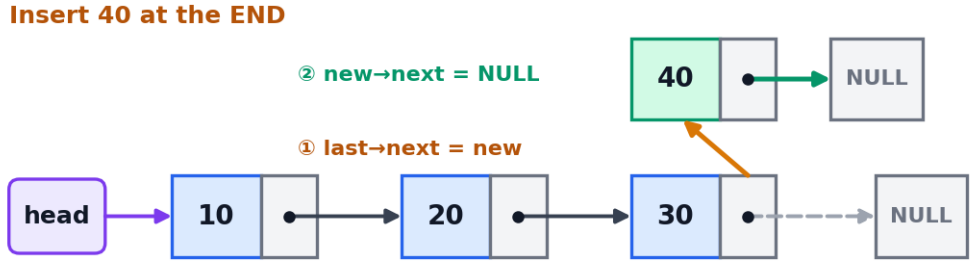


Fig: Insertion at the end – walk to the last node, then link

```
void insertEnd(int x) {
    struct Node *n = createNode(x), *temp = head;
    if (head == NULL) { head = n; return; }      // empty list
    while (temp->next != NULL) temp = temp->next; // go to the last node
    temp->next = n;                               // link last node to new node
}

void insertAfter(struct Node *p, int x) {        // insert after node p
    if (p == NULL) return;
    struct Node *n = createNode(x);
    n->next = p->next;     // step 1
    p->next = n;           // step 2
}
```

C. Deletion

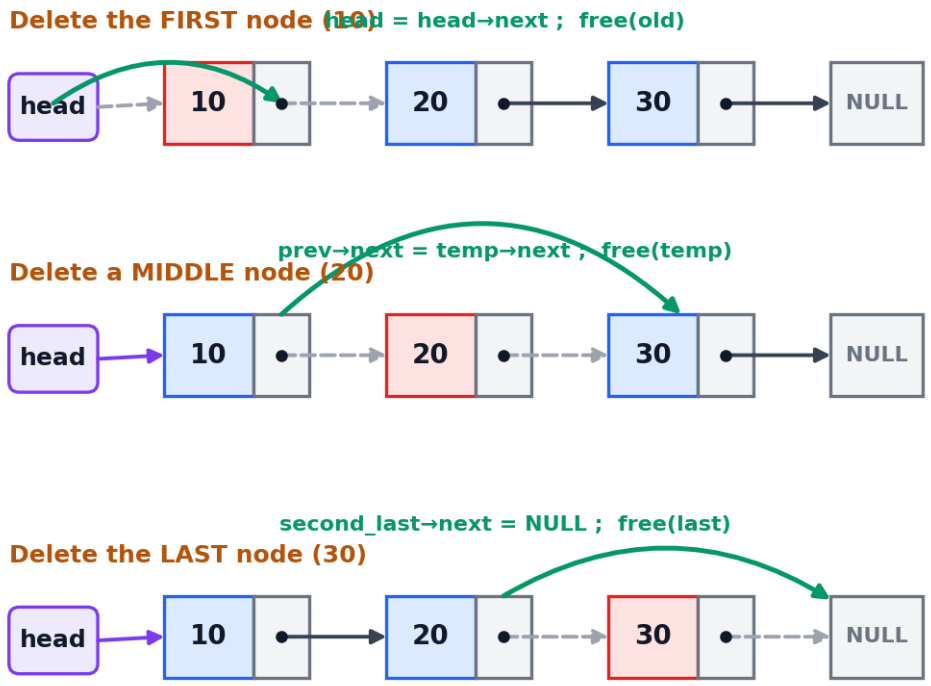


Fig: Deleting the first, a middle and the last node – dashed arrows are removed links

```
void deleteBegin() {
    if (head == NULL) { printf("Underflow"); return; }
    struct Node *t = head;  head = head->next;  free(t);
}
void deleteKey(int key) {                         // delete first node having value key
    struct Node *t = head, *prev = NULL;
    while (t != NULL && t->data != key) { prev = t; t = t->next; }
    if (t == NULL) return;                        // key not found
    if (prev == NULL) head = t->next;             // deleting the head
    else prev->next = t->next;                    // bypass the node
    free(t);
}
```

free() returns memory to the system – forgetting it causes a memory leak

D. Searching

```
int search(int key) {
    struct Node *t = head; int pos = 1;
    while (t != NULL) { if (t->data == key) return pos; t = t->next; pos++; }
    return -1;                    // not found      Time: O(n)
}
```

Worked example – tracing a list step by step

Step	Operation	List after the operation
1	insertBegin(30)	head → 30 → NULL
2	insertBegin(20)	head → 20 → 30 → NULL
3	insertEnd(40)	head → 20 → 30 → 40 → NULL
4	insertAfter(node 20, 25)	head → 20 → 25 → 30 → 40 → NULL
5	deleteKey(30)	head → 20 → 25 → 40 → NULL
6	deleteBegin()	head → 25 → 40 → NULL

E. Reversing a singly linked list

- Reverse the **direction of every link** – no data is moved.
- Use **three pointers**: **prev** (already reversed part), **curr** (node being handled), **next** (saved rest of list).
- Time **O(n)**, extra space **O(1)**.

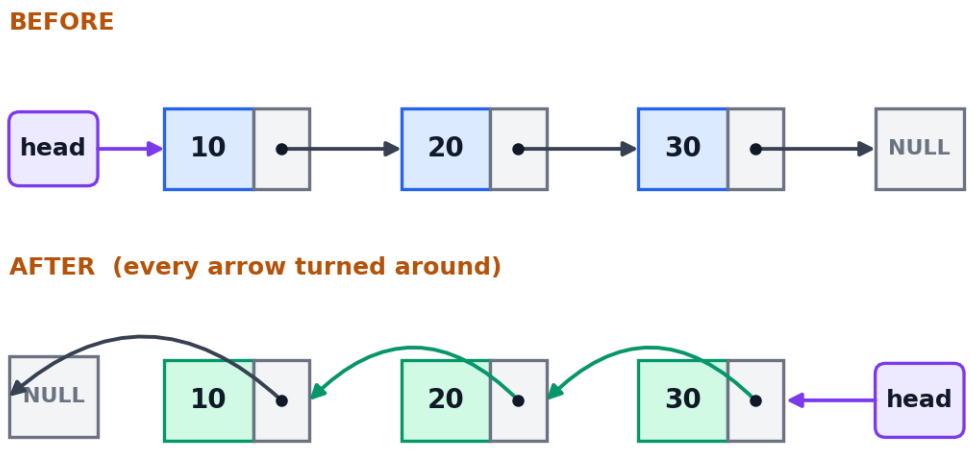


Fig: Reversing a list – last node becomes the new head

```
void reverse() {
    struct Node *prev = NULL, *curr = head, *next = NULL;
    while (curr != NULL) {
        next = curr->next;     // 1. save the rest of the list
        curr->next = prev;     // 2. turn the arrow around
        prev = curr;           // 3. move prev forward
        curr = next;           // 4. move curr forward
    }
    head = prev;               // prev is now the first node
}
```

COMMON MISTAKES

- Losing the **head** pointer = losing the whole list.
- Dereferencing a **NULL** pointer (e.g., temp→next when temp is NULL) crashes the program.
- Forgetting to **free()** deleted nodes causes memory leaks.
- Changing pointers in the **wrong order** disconnects the list.

Useful variants

- **Tail pointer** – keep a second pointer to the last node → insertion at the end becomes O(1).
- **Dummy (header) node** – a fake first node removes special cases for inserting / deleting at the head.
- **Finding the middle** – move **slow** by 1 and **fast** by 2; when fast reaches the end, slow is at the middle.

2.3 Doubly Linked List (DLL)

DEFINITION
Doubly linked list = each node has **three parts**: **prev** pointer, **data**, **next** pointer. It can be traversed in **both directions**.

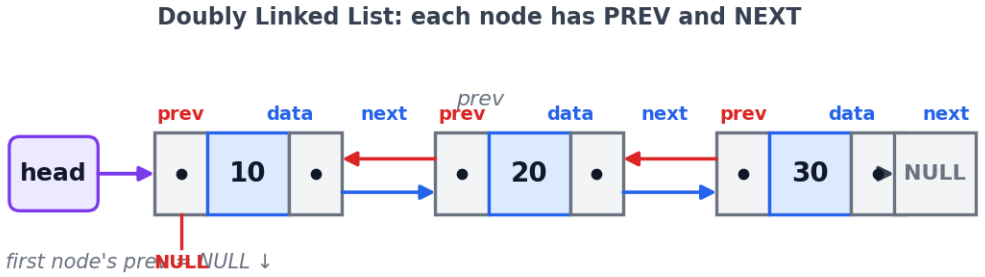


Fig: Doubly linked list – first node's prev and last node's next are NULL

```
struct DNode {
    struct DNode *prev;
    int data;
    struct DNode *next;
};
```

Advantages over singly linked list

- **Backward traversal** is possible.
- A node can be **deleted without a separate prev pointer** – the node itself knows its predecessor.
- Insertion / deletion before a given node is easy.

Disadvantages

- Extra memory for the **prev** pointer in every node.
- More pointers to update → **more chance of mistakes**.

Traversal in both directions

```
void forward(struct DNode *head) {                // first -> last
    for (struct DNode *t = head; t != NULL; t = t->next) printf("%d ", t->data);
}
void backward(struct DNode *tail) {               // last -> first (needs a tail pointer)
    for (struct DNode *t = tail; t != NULL; t = t->prev) printf("%d ", t->data);
}
```

Insertion between two nodes A and B

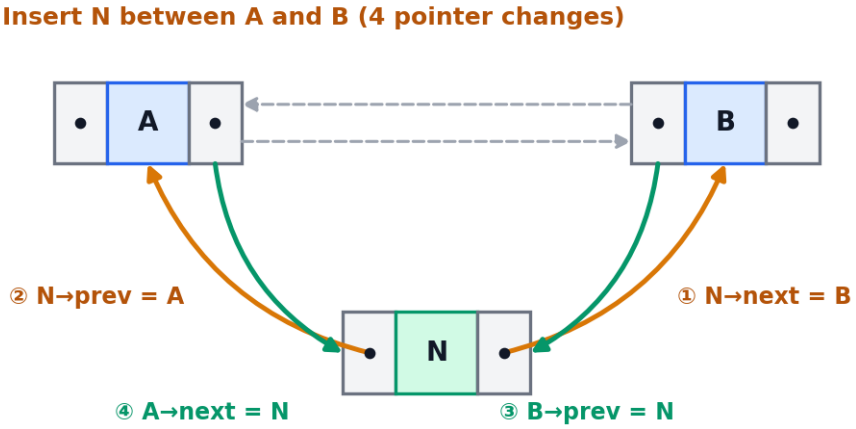


Fig: Four pointer changes are needed in a doubly linked list

```
void insertAfter(struct DNode *A, int x) {          // insert new node N after A
    struct DNode *N = (struct DNode*) malloc(sizeof(struct DNode));
    N->data = x;
    N->next = A->next;              // (1) N -> B
    N->prev = A;                    // (2) N <- A
    if (A->next != NULL) A->next->prev = N;   // (3) B -> prev = N
    A->next = N;                    // (4) A -> next = N
}
```

Deletion of a node X

1. **X→prev→next = X→next** (previous node skips X)
2. **X→next→prev = X→prev** (next node skips X) – only if X→next exists
3. **free(X)**

EXAM TIP
Special cases to mention in exams: deleting the **first** node (update head) and the **last** node (X→next is NULL). Always check for an **empty list**.

2.4 Circular Linked List (CLL)

DEFINITION
Circular linked list = a linked list in which the **last node points back to the first node**, so there is **no NULL** at the end.

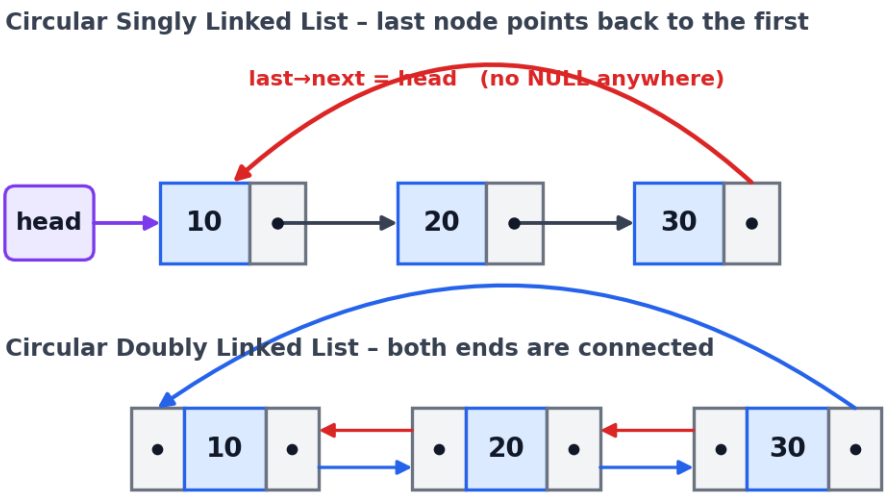


Fig: Circular singly and circular doubly linked lists

Key points

- Can be **singly** or **doubly** circular.
- From **any node** you can reach every other node.
- Traversal stops when we come **back to the starting node** – NOT at NULL.
- If a pointer **tail** to the last node is kept, insertion at both ends is **O(1)** (head = tail→next).
- Danger: using “while (temp != NULL)” gives an **infinite loop**.

```
void displayCircular() {
    if (head == NULL) return;
    struct Node *t = head;
    do {
        printf("%d ", t->data);
        t = t->next;
    } while (t != head);          // stop when we are back at the start
}

void insertEndCircular(int x) {   // singly circular
    struct Node *n = createNode(x);
    if (head == NULL) { head = n; n->next = head; return; }
    struct Node *t = head;
    while (t->next != head) t = t->next;   // find last node
    t->next = n;   n->next = head;         // keep the circle closed
}
```

Deletion in a circular list (singly)

- **Only one node:** free it and set head = NULL.
- **Deleting the first node:** find the last node, set last→next = head→next, move head, free the old head.
- **Deleting any other node:** same as in a singly list – prev→next = temp→next.

Basis	Singly	Doubly	Circular
Pointers / node	1	2	1 or 2
Traversal	Forward	Both ways	Round and round
End marker	NULL	NULL	Link to first node
Extra memory	Least	Most	Same as base type
Best for	Simple dynamic data	Back / forward moves	Repeating cycles

2.5 Arrays vs Linked Lists

Basis	Array	Linked list
Memory allocation	Static (compile time), fixed size	Dynamic (run time), grows / shrinks
Memory layout	Contiguous	Scattered, joined by pointers
Access of element	**Direct** a[i] → O(1)	Sequential → O(n)
Insertion / deletion	Costly – shifting, O(n)	**Easy** – O(1) once position is known
Extra memory	None	Pointer(s) in every node
Memory wastage	Possible (unused slots)	No unused slots, but pointer overhead
Searching	Binary search possible (sorted)	Only linear search
Cache performance	Good (neighbouring data)	Poor
Use when	Size known, frequent random access	Size unknown, frequent insert / delete

KEY IDEA
Rule of thumb: need quick **access by index** → choose an **array**. Need frequent **insertion / deletion** and the size keeps changing → choose a **linked list**.

2.6 Applications of Linked Lists



Fig: Typical applications grouped by type of list

Polynomial representation

Each term is stored as a node: **coefficient | exponent | next**. Adding two polynomials = merge the two lists by exponent. No space is wasted for missing terms (sparse polynomial).

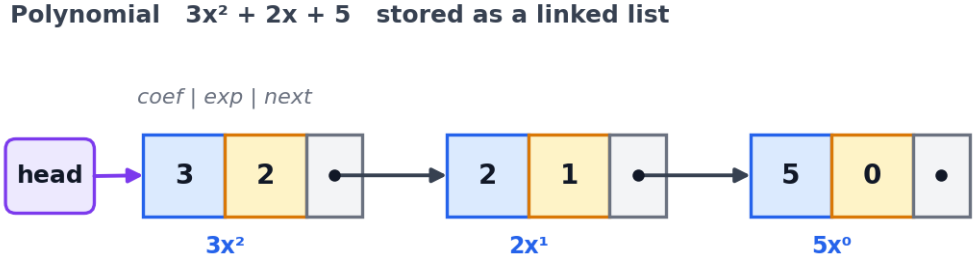


Fig: Polynomial 3x² + 2x + 5 using a linked list

Example – round-robin CPU scheduling (circular list)

Processes P1 → P2 → P3 form a circle. The CPU gives each process a **time slice** and then moves to the next node. A finished process is deleted from the circle; the loop continues until the list is empty.

Time slice	CPU runs	Circle after the slice
1	P1	P1 → P2 → P3 → (P1)
2	P2 (finishes)	P1 → P3 → (P1)
3	P3	P1 → P3 → (P1)
4	P1 (finishes)	P3 → (P3)

More applications

- **Implementation of stacks and queues** (dynamic, no overflow until memory ends).
- **Dynamic memory management** – operating system keeps a linked list of free blocks.
- **Hash table chaining** – colliding keys are stored in a list (Unit V).
- **Graph adjacency lists** – list of neighbours of each vertex.
- **Undo / redo** and **browser history** – doubly linked list of states / pages.
- **Music or image playlist** – next / previous (DLL) and repeat-all (circular list).
- **Round-robin scheduling** – processes arranged in a circular list; CPU moves to the next after every time slice.
- **Sparse matrix** and **big-number arithmetic** – store only non-zero parts / digits.

2.7 Quick Revision & Practice

ONE-PAGE SUMMARY

- Linked list = **nodes** (data + pointer) joined by links; head = entry point; NULL = end.
- SLL: insert at beginning O(1); end O(n); delete last O(n); search O(n).
- DLL: prev + next → both directions, 4 pointer changes on insertion.
- CLL: last → first; loop with **do-while (t != head)**.
- Array → fast access, slow insert/delete. Linked list → slow access, fast insert/delete.
- Always **update pointers in the right order** and **free()** deleted nodes.

Likely exam questions

2 marks

- What is a self-referential structure?
- Define a node. What are its parts?
- Mention two advantages of a doubly linked list.
- What is a circular linked list?
- Why is binary search not suitable for linked lists?

5 marks

- Write an algorithm to insert a node at the beginning and at the end of a singly linked list.
- Explain deletion of a node from a doubly linked list with a diagram.
- Compare arrays and linked lists.
- Explain the representation of a polynomial using a linked list.

10 marks

- Explain all operations (create, insert, delete, search, display) on a singly linked list with C code.
- Explain singly, doubly and circular linked lists with diagrams and applications.

Try it yourself

1. Draw the list after: insertBegin(30), insertBegin(20), insertEnd(40), insertAfter(20, 25).
2. Write a function to **count the nodes** of a singly linked list.
3. Write a function to **reverse** a singly linked list using three pointers (prev, curr, next).
4. Find the **middle** node in one pass using two pointers (slow and fast).